

Matrices and Arrays

MATLAB is an abbreviation for "matrix laboratory." While other programming languages mostly work with numbers one at a time, MATLAB® is designed to operate primarily on whole matrices and arrays.

All MATLAB variables are multidimensional *arrays*, no matter what type of data. A *matrix* is a two-dimensional array often used for linear algebra.

[Open in MATLAB Online](#)
[Copy Command](#)

Array Creation

To create an array with four elements in a single row, separate the elements with either a comma (,) or a space.

```
a = [1 2 3 4]
```

[Get](#) ▼

```
a = 1×4
```

1	2	3	4
---	---	---	---

This type of array is a *row vector*.

To create a matrix that has multiple rows, separate the rows with semicolons.

```
a = [1 3 5; 2 4 6; 7 8 10]
```

[Get](#) ▼

```
a = 3×3
```

1	3	5
2	4	6
7	8	10

You can also define each row on its own line of code and separate the rows with a newline.

```
a = [1 3 5
     2 4 6
     7 8 10]
```

[Get](#) ▼

```
a = 3×3
```

1	3	5
2	4	6
7	8	10

Another way to create a matrix is to use a function, such as `ones`, `zeros`, or `rand`. For example, create a 5-by-1 column vector of zeros.

```
z = zeros(5,1)
```

[Get](#) ▼

```
z = 5×1
```

0
0

0
0
0

Matrix and Array Operations

MATLAB allows you to process all of the values in a matrix using a single arithmetic operator or function.

```
a + 10
```

 Get ▼

```
ans = 3×3
```

```
11    13    15
12    14    16
17    18    20
```

```
sin(a)
```

 Get ▼

```
ans = 3×3
```

```
0.8415    0.1411   -0.9589
0.9093   -0.7568   -0.2794
0.6570    0.9894   -0.5440
```

To transpose a matrix, use a single quote ('):

```
a'
```

 Get ▼

```
ans = 3×3
```

```
1     2     7
3     4     8
5     6    10
```

You can perform standard matrix multiplication, which computes the inner products between rows and columns, using the * operator. For example, confirm that a matrix times its inverse returns the identity matrix:

```
p = a*inv(a)
```

 Get ▼

```
p = 3×3
```

```
1.0000    0.0000   -0.0000
0         1.0000   -0.0000
0         0.0000    1.0000
```

Notice that p is not a matrix of integer values. MATLAB stores numbers as floating-point values, and arithmetic operations are sensitive to small differences between the actual value and its floating-point representation. You can display more decimal digits using the format command:

```
format long
```

 Get ▼

```
p = a*inv(a)
```

```
p = 3×3
```

```
0.999999999999996    0.000000000000007    -0.000000000000002
                   0    1.000000000000000    -0.000000000000003
                   0    0.000000000000014    0.999999999999995
```

Reset the display to the shorter format using

```
format short
```

 Get ▼

format affects only the display of numbers, not the way MATLAB computes or saves them.

To perform element-wise multiplication rather than matrix multiplication, use the `.*` operator:

```
p = a.*a
```

 Get ▼

```
p = 3×3
```

```
    1     9    25
    4    16    36
   49    64   100
```

The matrix operators for multiplication, division, and power each have a corresponding array operator that operates element-wise. For example, raise each element of `a` to the third power:

```
a.^3
```

 Get ▼

```
ans = 3×3
```

```
     1     27    125
     8     64    216
    343    512   1000
```

Concatenation

Concatenation is the process of joining arrays to make larger ones. In fact, you made your first array by concatenating its individual elements. The pair of square brackets `[ ]` is the concatenation operator.

```
A = [a,a]
```

 Get ▼

```
A = 3×6
```

```
    1     3     5     1     3     5
    2     4     6     2     4     6
    7     8    10     7     8    10
```

Concatenating arrays next to one another using commas is called *horizontal* concatenation. Each array must have the same number of rows. Similarly, when the arrays have the same number of columns, you can concatenate *vertically* using semicolons.

```
A = [a; a]
```

 Get ▼

```
A = 6×3
```

```
    1     3     5
    2     4     6
    7     8    10
```

1	3	5
2	4	6
7	8	10

Complex Numbers

Complex numbers have both real and imaginary parts, where the imaginary unit is the square root of -1.

```
sqrt(-1)
```

[Get](#) ▼

```
ans =  
0.0000 + 1.0000i
```

To represent the imaginary part of complex numbers, use either `i` or `j`.

```
c = [3+4i, 4+3j; -i, 10j]
```

[Get](#) ▼

```
c = 2x2 complex
```

```
3.0000 + 4.0000i    4.0000 + 3.0000i  
0.0000 - 1.0000i    0.0000 +10.0000i
```

Get Started with MATLAB



NEXT >

YOU MIGHT ALSO BE INTERESTED IN

